arXiv mixed with Tableau-Style Report

Faked using R+Quarto+LaTeX code hidden

for Stats 422

Abstract

This is the abstract. It should be a single paragraph summarizing the methodology and results. It will appear centered.

Introduction

We have reached the end. We hope that you have sufficient tools to write a report with the styling you desire. There are two versions of this (they are the same) the second one reveals the code for teaching purposes

```
library(ggplot2)
library(ggthemes)
library(showtext)
library(dplyr)
library(lubridate)
library(tidyr)
library(knitr) # for kable()
library(sf)
library(rnaturalearth)
library(rnaturalearthdata)
library(scales)
library(ggdag)
library(NeuralNetTools) # to visualize NN
library(nnet)

# Load any Google Fonts you want to use
#    first argument = Google Font Name
#    second argument = the name you will use in R
font_add_google("Spectral", "spectral")
font_add_google("Roboto", "roboto")

# Turn on showtext to render the fonts automatically
showtext_auto()

# Set the default theme to Tufte for a high data-to-ink ratio.
# Add some specific options to make the appearance cleaner.
theme_set(
  theme_tufte(base_size = 11, base_family = "Spectral") +
```

```
    theme(
      text = element_text(color = "#333333"),

      plot.title = element_text(face = "bold", size = 14, hjust = 0),
      plot.subtitle = element_text(color = "#666666", size = 10, hjust = 0, margin = margin(b =
      plot.caption = element_text(color = "#888888", size = 8, hjust = 1),

      # Remove almost all background elements for cleanness
      panel.background = element_rect(fill = "white", color = NA),
      plot.background = element_rect(fill = "white", color = NA),

      # Keep only essential axis lines
      axis.line = element_line(color = "#333333", size = 0.2),
      axis.ticks = element_line(color = "#333333", size = 0.2),

      # Legends at the top or bottom only
      legend.position = "top",
      legend.justification = "left",
      legend.title = element_text(face = "bold", size = 9),
      legend.text = element_text(size = 9)
    )
)

# Set default color scales to Tableau for that "look"

options(
  ggplot2.discrete.colour = ggthemes::scale_color_tableau,
  ggplot2.discrete.fill = ggthemes::scale_fill_tableau
)
```

Lorem ipsum filler

```
lorem::ipsum(paragraphs = 4, avg_words_per_sentence = 15)
```

Adipiscing duis mi, urna ornare, parturient lectus, ultricies libero imperdiet. Malesuada augue imperdiet eleifend lobortis lobortis faucibus a egestas, curabitur eu, metus massa ante, viverra blandit lectus potenti, augue convallis, suscipit: euismod, senectus iaculis porttitor penatibus volutpat interdum?

Amet porttitor posuere dui porttitor mus pellentesque suspendisse sed. Hendrerit vehicula porttitor sed faucibus ad convallis etiam! Fames congue eleifend, ligula enim – dictum lobortis curae cursus dapibus porttitor. Malesuada sodales a primis et in. Augue class torquent ut magna velit mi sem justo varius turpis nascetur volutpat auctor potenti cubilia mi donec platea, ut aliquet per vulputate rhoncus netus, lobortis, integer turpis, taciti; donec a pulvinar eu pharetra imperdiet bibendum malesuada, congue dictumst morbi nibh id facilisis gravida neque euismod habitant aptent accumsan purus erat fusce fusce nullam massa habitasse.

Amet lacinia platea nec. Proin tristique eget vitae mauris ante posuere fermentum nascetur conu-

bia nunc? Proin fames elementum nec per arcu, tempus lacinia placerat interdum platea pharetra. Mollis magna conubia luctus nec potenti ad etiam odio donec, taciti bibendum, consequat luctus consequat, enim, augue mi, etiam cum mauris natoque vulputate congue cubilia nascetur volutpat facilisi ridiculus bibendum nascetur cum natoque bibendum nulla.

Lorem lacus euismod, porta feugiat urna penatibus vulputate ac tincidunt. Nisl phasellus quam ac nostra cursus iaculis: arcu: venenatis, faucibus feugiat torquent orci. Urna mauris vel aliquet ad erat varius praesent etiam; netus nostra lobortis laoreet torquent; ad, habitant hac posuere habitant ullamcorper, ad fusce metus neque praesent lectus commodo vulputate vestibulum.

Bar Chart

The humble bar chart/bar plot is one of the most effective tools in data visualization. Use them to compare values across different categories with rectangular bars where the length of the bar is proportional to the value it represents. Recall the human processes the meaning most correctly.

```
# Sample Data
data <- tibble(
  Region = c("North", "South", "East", "West"),
  Revenue = c(450000, 620000, 310000, 550000)
)

# Create the ggplot2 bar chart
tableau_bar_chart <- data %>%
  # Sort data for a better visual comparison
  arrange(desc(Revenue)) %>%
  ggplot(aes(x = reorder(Region, Revenue), y = Revenue, fill = Region)) +

  # Bar layer
  geom_col(width = 0.7) +

  # Remove axis ticks and lines common in clean styles
  theme(
    # Remove vertical grid lines
    axis.line.y = element_blank(),
    axis.ticks.y = element_blank(),

    # Hide the y-axis title
    axis.title.y = element_blank(),

    # Place labels directly on bars
    legend.position = "none"
  ) +

  # Add direct labels to the bars
  geom_text(
    aes(label = scales::dollar(Revenue, prefix = "$", accuracy = 1)),
    hjust = -0.1,
    size = 4
  ) +

  # Scale and coordinate adjustments
  scale_y_continuous(
    labels = scales::dollar_format(prefix = "$", scale = 0.001, suffix = "K"),
    expand = expansion(mult = c(0, 0.2)) # Give space for labels on the right
  ) +

  # Flip coordinates for a horizontal bar chart
```

```
  coord_flip() +

  # Set titles and labels
  labs(
    x = NULL, # Remove the x-axis label after coord_flip()
    title = "Regional Sales Performance",
    subtitle = "Annual Revenue in Thousands"
  )

# Print the chart
tableau_bar_chart
```

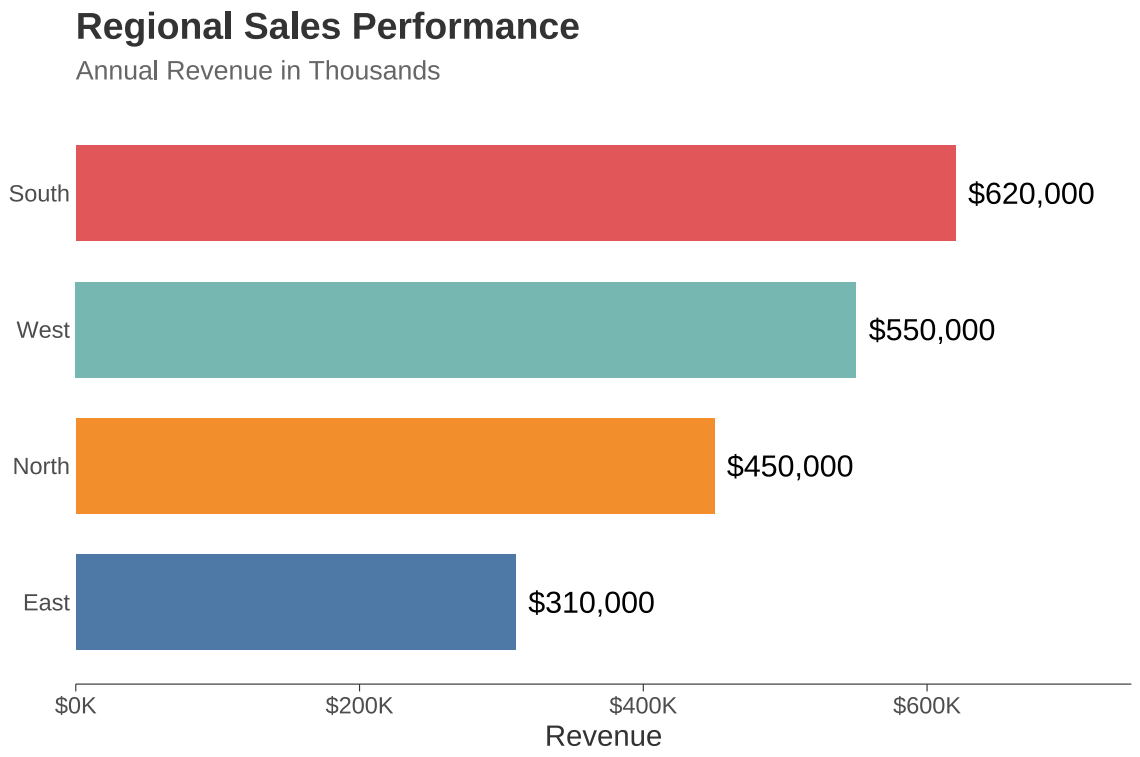


Figure 1: Revenue by Region

Line Plot

Line plots communicate change over time more clearly than any other chart type. Use them to show trends, the shape of change and to compare with other time lines.

```
# Simulated Monthly Traffic Data
traffic_data <- tibble(
  Month = seq.Date(from = as.Date("2024-01-01"), by = "month", length.out = 12),
  Unpaid = round(5000 + cumsum(rnorm(12, 500, 100))),
  Paid = round(3000 + cumsum(rnorm(12, 300, 50)))
)

# Reshape data to long format for easy plotting
long_data <- traffic_data %>%
  tidyr::pivot_longer(cols = c(Unpaid, Paid), names_to = "Source", values_to = "Visitors")

# Create the Line Plot
tableau_line_chart <- ggplot(long_data, aes(x = Month, y = Visitors, color = Source)) +

  # Add lines
  geom_line(linewidth = 1) +

  # Add points for clarity
  geom_point(size = 2) +

  # Adjust theme elements for a cleaner look
  theme(
    panel.grid.minor = element_blank(), # Remove minor gridlines
    panel.grid.major.x = element_blank(), # Keep only horizontal major gridlines
    legend.position = "top"
  ) +

  # Format axes and labels
  scale_y_continuous(labels = scales::comma) +
  scale_x_date(date_breaks = "2 months", date_labels = "%b %y") +

  labs(
    title = "Website Traffic Trend by Source",
    x = NULL,
    y = "Total Visitors"
  )

tableau_line_chart
```

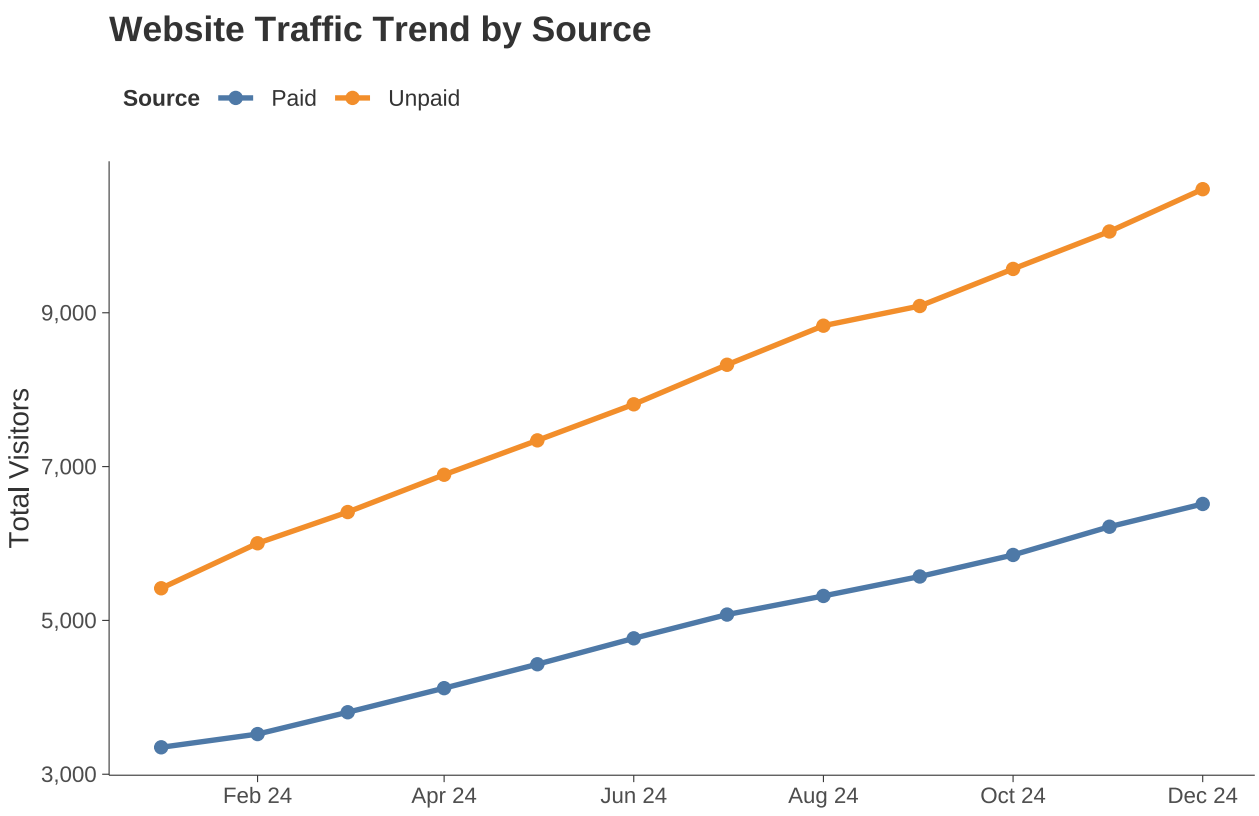


Figure 2: Monthly Website Traffic by Source

Scatterplot

This is the classic for visualizing the relationship between two continuous variables. Even more complex or modern graphics (like UMAP below)

```
# Simulated Customer Data
set.seed(42)
customer_data <- tibble(
  Resolution_Time_Hours = runif(100, 1, 48),
  Satisfaction_Score = 10 - 0.15 * Resolution_Time_Hours + rnorm(100, 0, 1),
  Priority = sample(c("Low", "Medium", "High"), 100, replace = TRUE, prob = c(0.4, 0.4, 0.2))
)

# Create the Scatter Plot
tableau_scatter_chart <- ggplot(customer_data,
                                aes(x = Resolution_Time_Hours,
                                    y = Satisfaction_Score,
                                    color = Priority)) +

  # Add points, adjusted for clarity
  geom_point(alpha = 0.8, size = 3) +

  # Add a trend line for the overall data
  geom_smooth(method = "lm", se = FALSE, color = "darkgray", linetype = "dashed") +

  # Set axis labels and titles
  labs(
    title = "Satisfaction Score vs. Ticket Resolution Time",
    subtitle = "Points colored by Ticket Priority",
    x = "Resolution Time (Hours)",
    y = "Satisfaction Score (1-10)"
  )

tableau_scatter_chart
```

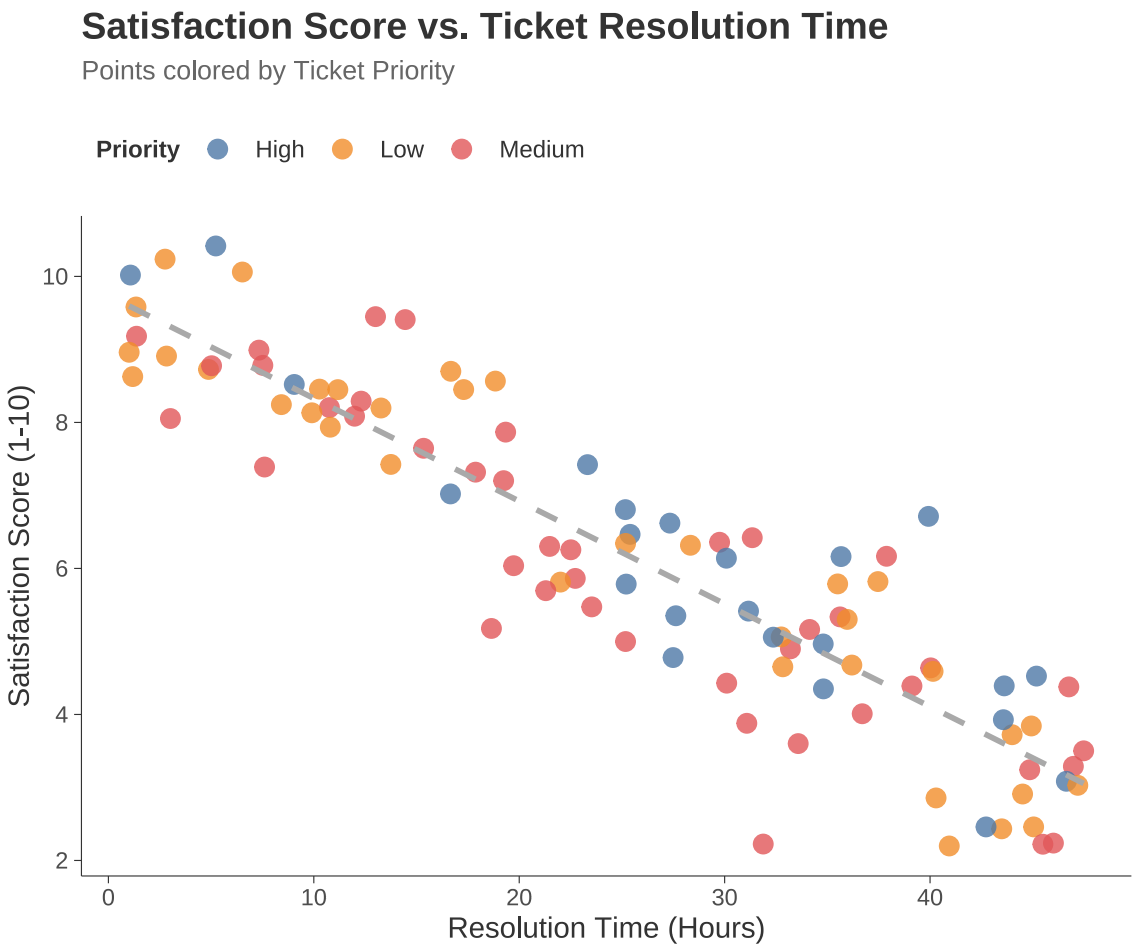


Figure 3: Customer Satisfaction vs. Ticket Resolution Time

Box plot

Boxplots reveal a lot of distributional information in a compact, standardized way and they are easy to compare. Typically combining categorical and continuous information in a single visual.

```
# Simulated Latency Data
set.seed(123)
latency_data <- tibble(
  Latency_ms = c(
    rnorm(100, 50, 10),  # DC-A (Low)
    rnorm(100, 75, 15),  # DC-B (Medium)
    rnorm(100, 40, 5)    # DC-C (Very Low)
  ),
  Data_Center = factor(c(rep("DC-A (US)", 100), rep("DC-B (EU)", 100), rep("DC-C (Asia)", 100))
)

# Create the Box Plot
tableau_box_chart <- ggplot(latency_data, aes(x = Data_Center, y = Latency_ms, fill = Data_Cent

  # Add the box plot layer
  geom_boxplot(width = 0.6, alpha = 0.8) +

  # Add jittered points (optional, but shows underlying data density)
  geom_jitter(color = "black", size = 0.5, alpha = 0.6) +

  # Set axis labels and titles
  labs(
    title = "Latency Distribution by Data Center",
    x = NULL, # Remove X-axis label as the categories are self-explanatory
    y = "Latency (Milliseconds)"
  )

tableau_box_chart
```

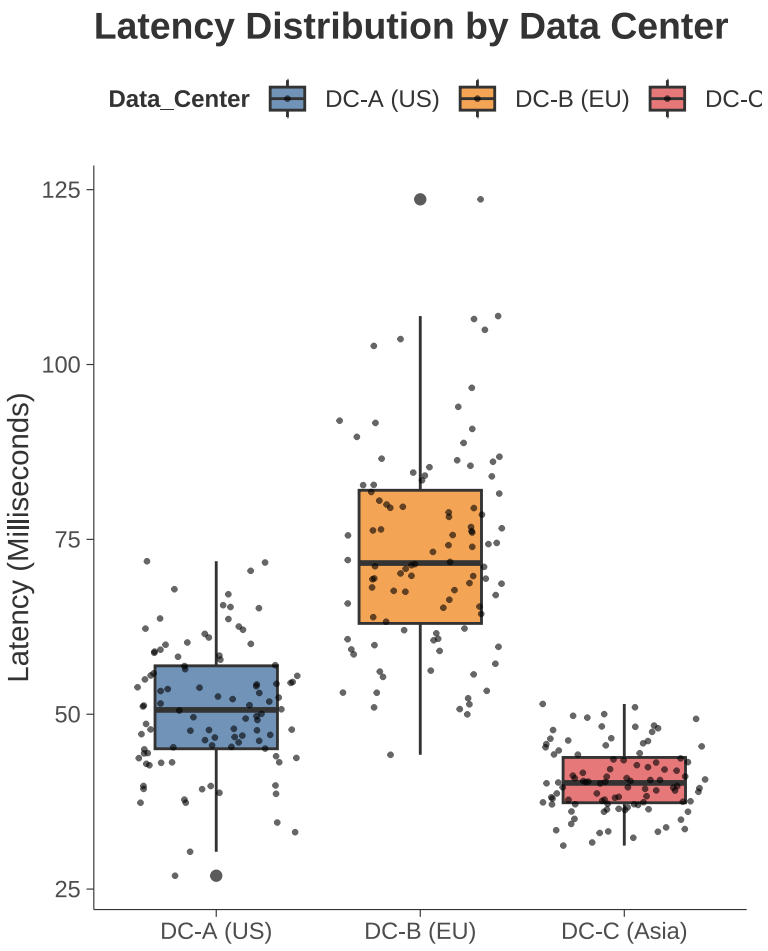


Figure 4: Distribution of Server Latency by Data Center

Mapping

We spent more time than you probably wanted on mapping, at its simplest, we can use various mapping libraries in R, this first one uses `rnaturalearth` but simpler ones that require fewer resources are like `maps`

```
# Load World data
world <- ne_countries(scale = "medium", returnclass = "sf")

# Simulate a variable (e.g., Population Density)
world <- world %>%
  mutate(pop_density = log(pop_est / sqrt(gdp_md)))

# Create the Themed Map
tableau_map <- ggplot(data = world) +
  # Use geom_sf for spatial features, filling by our variable
  geom_sf(aes(fill = pop_density), color = "gray80", linewidth = 0.1) +

  # Set a clean, Tableau-like color palette
  scale_fill_viridis_c(option = "magma", name = "Log Population Density") +

  labs(title = "Global Analysis with Minimalist Aesthetic")

tableau_map
```

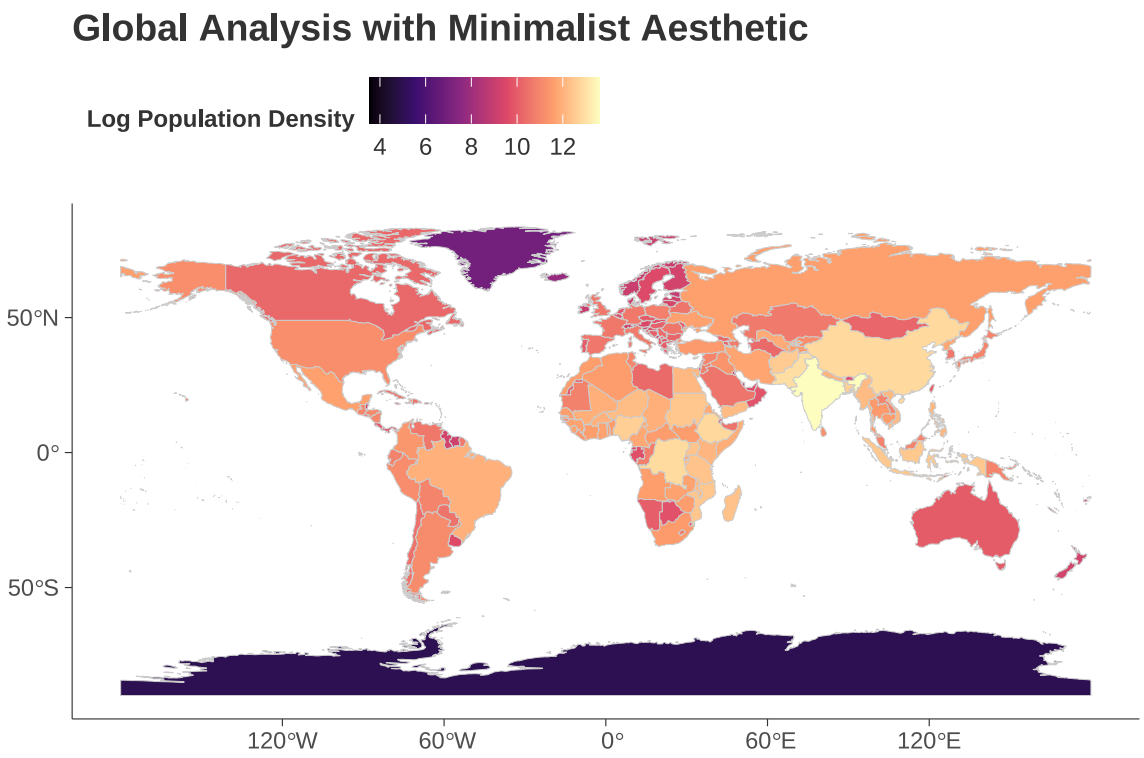


Figure 5: Themed Map of World Population Density

Heat Map

A slightly different map. Heatmaps are the go-to when your data are structured as a matrix so correlation, distance, confusion matrices, but also gene expression data

```
# Simulated Performance Data (e.g., Sales Margin %)
performance_data <- expand.grid(
  Quarter = factor(paste0("Q", 1:4), levels = paste0("Q", 1:4)),
  Product = factor(paste0("Product ", LETTERS[1:5]))
) %>%
  mutate(
    Margin = round(runif(20, 0.1, 0.35), 3),
    Label = scales::percent(Margin, accuracy = 0.1)
  )

# Create the Heat Map
tableau_heatmap <- ggplot(performance_data, aes(x = Quarter, y = Product, fill = Margin)) +

  # Tile layer for the heat map
  geom_tile(color = "white", linewidth = 1) +

  # Text labels for the values (critical for Tableau style)
  geom_text(aes(label = Label), color = "black", size = 4) +
  # **APPLYING TABLEAU SEQUENTIAL COLOR SCALE**
  # 'Blue' or 'Orange-Blue Diverging' are common Tableau options.
  scale_fill_gradient(
    low = "#EBF5FF", high = "#4E79A7", # Tableau blue gradient (Tableau-like)
    name = "Margin %",
    labels = scales::percent
  ) +

  labs(
    title = "Product Margin Performance by Quarter",
    subtitle = "Higher contrast blue gradient emphasizes top performance"
  )

tableau_heatmap
```

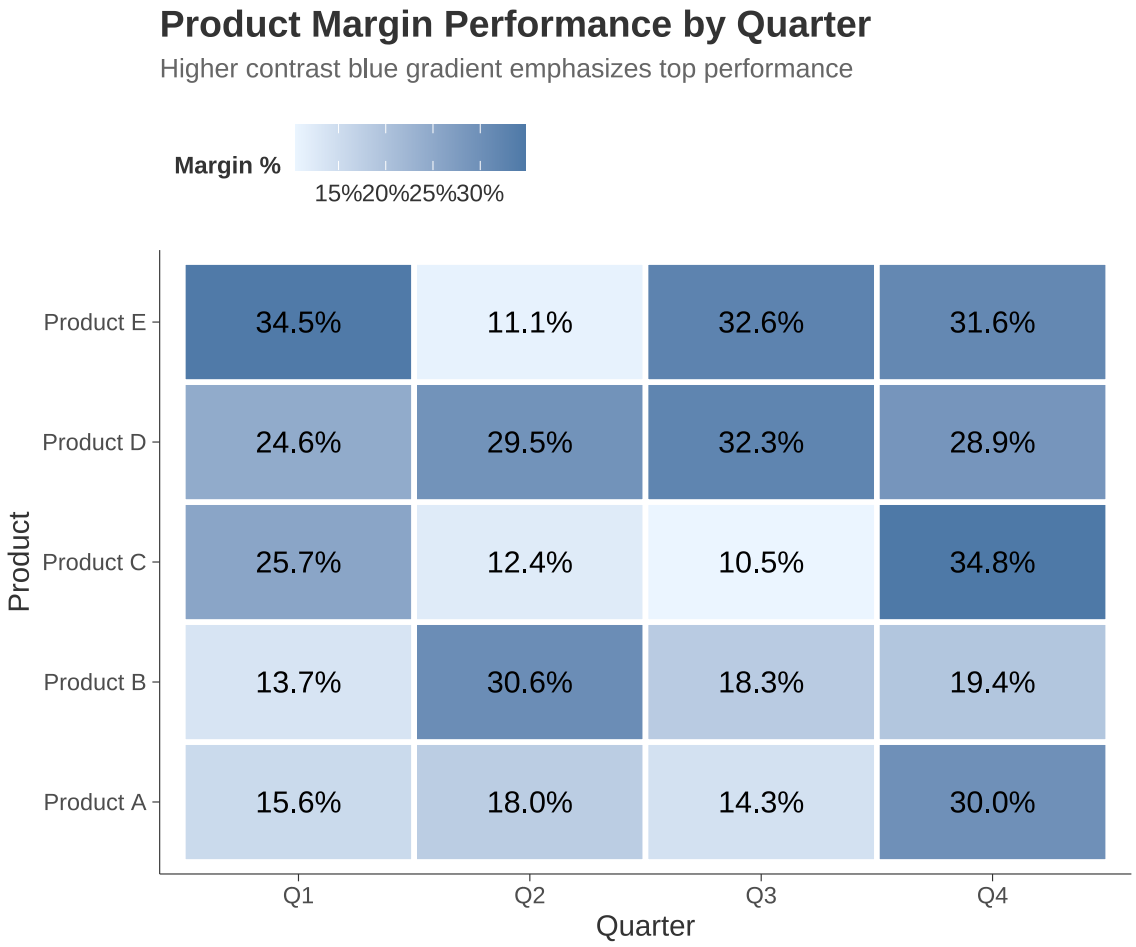


Figure 6: Quarterly Performance Heat Map (Tableau Sequential Color)

More Advanced/Newer

You do not typically see these in an undergraduate curriculum.

UMAP

UMAP (Uniform Manifold Approximation and Projection) is used for dimension reduction in data visualization and machine learning.

UMAP takes a dataset with many features (dimensions) and collapses into a 2D or 3D scatter plot. This allows us to identify clusters, patterns, and relationships that would be impossible to see in data with hundreds of columns.

UMAP areas of application

Genomics, Image Processing: Grouping similar images (e.g., separating photos of cats, dogs, and cars) based on pixel data. NLP – Visualizing word embeddings (grouping words with similar meanings together).

You can check out a cute video on UMAP: https://www.youtube.com/watch?v=eN0wFzBA4Sc&t=12s

```
set.seed(123)
umap_data <- tibble(
  UMAP1 = c(rnorm(50, 2, 0.5), rnorm(50, 5, 0.5), rnorm(50, 2, 0.5)),
  UMAP2 = c(rnorm(50, 2, 0.5), rnorm(50, 2, 0.5), rnorm(50, 5, 0.5)),
  Cluster = factor(c(rep("A", 50), rep("B", 50), rep("C", 50)))
)

kable(head(umap_data))
```

UMAP1	UMAP2	Cluster
1.719762	2.393869	A
1.884911	2.384521	A
2.779354	2.166101	A
2.035254	1.495812	A
2.064644	1.940274	A
2.857533	1.859802	A

```
# Create the Themed UMAP Scatter Plot
tableau_umap <- ggplot(umap_data, aes(x = UMAP1, y = UMAP2, color = Cluster)) +

  geom_point(alpha = 0.7, size = 2.5) +

# STRIP AXIS TEXT AND TICKS
  theme(
    axis.text = element_blank(),  # Removes the numbers (2, 4, 6)
    axis.ticks = element_blank(), # Removes the little tick marks
    panel.grid = element_blank()  # Ensure no grid lines distract from clusters
  ) +
```

```
  labs(
    title = "Data Clusters in UMAP Space",
    # Keep axis titles so reader knows what dimensions these are
    x = "UMAP Dimension 1",
    y = "UMAP Dimension 2"
  )

tableau_umap
```

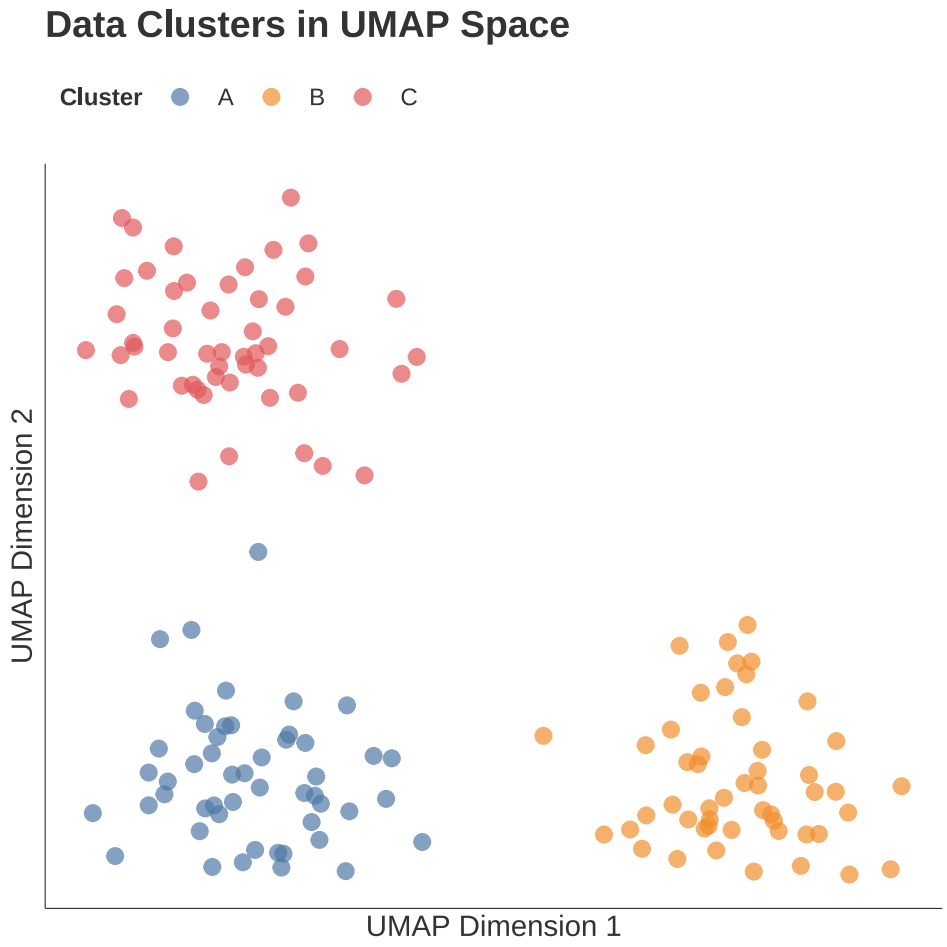
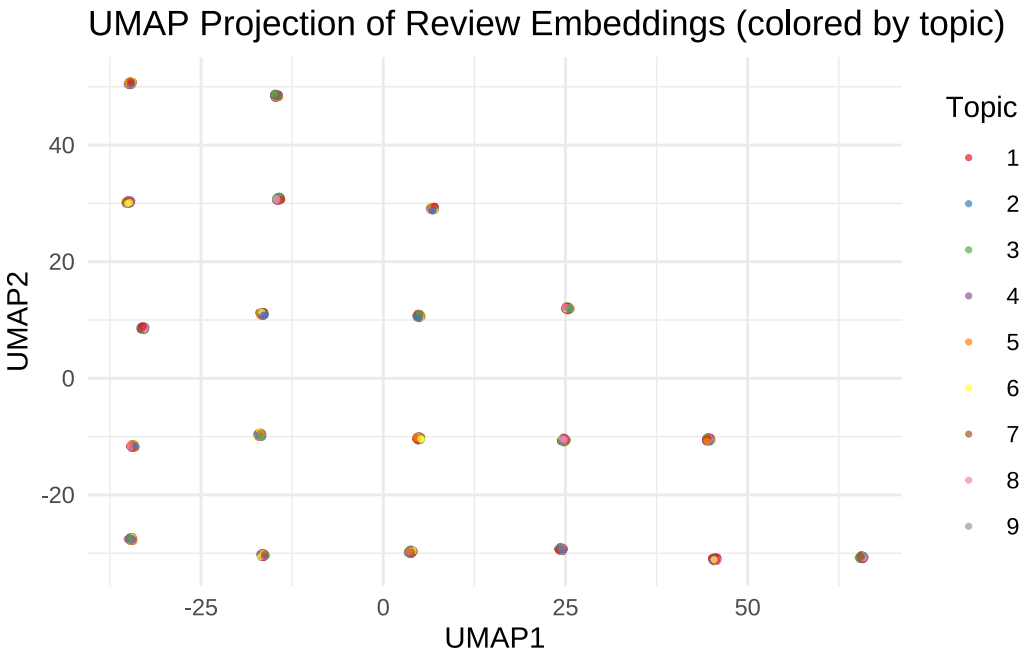


Figure 7: UMAP Projection of Simulated High-Dimensional Data

A Second UMAP Example

Slightly different data (a little large, you can view it separately)

```
library(umap)
library(ggplot2)
library(dplyr)

set.seed(222)

# Fake data for 6,000 reviews, 9 topics
n_samples <- 6000
n_topics  <- 9
topics <- sample(1:n_topics, n_samples, replace = TRUE)

# Create mock 300-D embeddings (vectors) with small topic shifts
embeddings <- matrix(rnorm(n_samples * 300), nrow = n_samples) +
              matrix(rep(topics, each = 300), nrow = n_samples) * 0.5

# Reduce to 2D with UMAP
umap_res <- umap(embeddings, n_neighbors = 15, min_dist = 0.2, metric = "cosine")
df_umap <- data.frame(UMAP1 = umap_res$layout[,1],
                      UMAP2 = umap_res$layout[,2],
                      Topic = factor(topics))

# Plot it
ggplot(df_umap, aes(UMAP1, UMAP2, color = Topic)) +
  geom_point(size = 0.6, alpha = 0.7) +
  scale_color_brewer(palette = "Set1") +
  labs(title = "UMAP Projection of Review Embeddings (colored by topic)") +
  theme_minimal() +
  theme(legend.position = "right")
```


UMAP Projection of Review Embeddings (colored by topic)
UMAP2
40
20
0
-20
-25
0
25
50
UMAP1
Topic
1
2
3
4
5
6
7
8
9

Neural Net

The code below trains a neural network to classify iris flowers based on their measurements, and then it creates a visualization of that network's architecture and weights.

```
# A simple NN model
set.seed(867)
iris_nn <- nnet(Species ~ ., data = iris, size = 3, decay = 5e-4, maxit = 200, trace = FALSE)
```

The line above uses the nnet package to build a neural network with a single hidden layer. Measurements from iris are pushed forward through 3 hidden processing units, and immediately outputs a classification, without ever looping back to reconsider or "remember" previous data. You can get a better idea see: https://www.lxt.ai/ai-glossary/neural-networks/

Visualize the network using the NeuralNetTools function plotnet

```
plotnet(iris_nn, y_names = c("Setosa", "Versicolor", "Virginica"), cex_val=0.6, circle_col = "
```

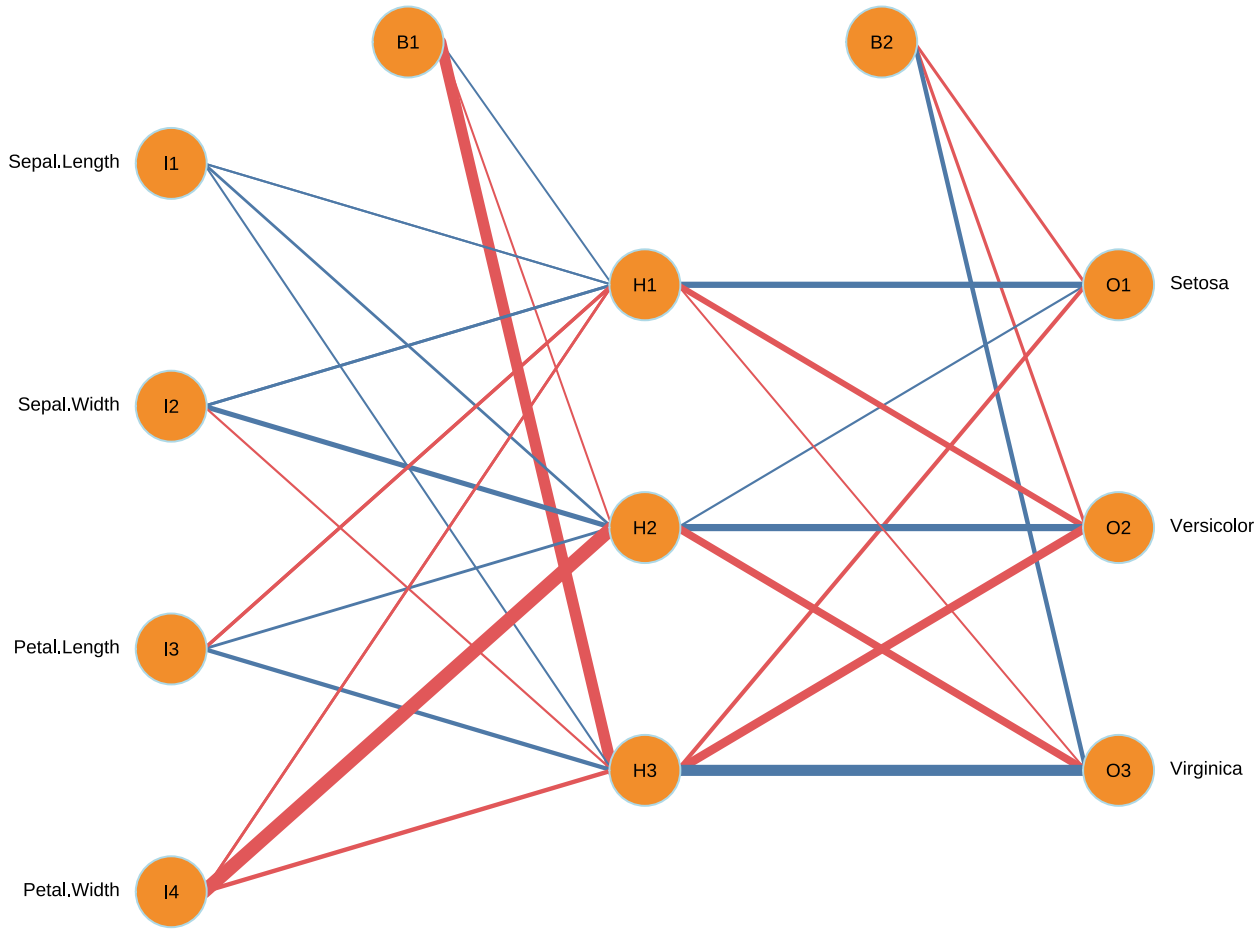


Figure 8: Neural Network Architecture

Network Data generally

We can find network data anywhere, the easiest form to use contains pairs which can serve as nodes. This is data from the Tax Treaties explorer (https://www.treaties.tax/en/)

```
library(tidyverse)
library(igraph)
library(ggraph)
library(maps) # for map background

# Load and Prepare Network Data
raw_data <- read_csv("./tax_treaty_data.csv")

kable(head(raw_data[, 1:5]))
```

Country A	Country B	Type	Date of signature	Effective
Australia	Kiribati	Original	1991	1991
Australia	Philippines	Original	1979	1980
Australia	Sri Lanka	Original	1989	1992
Argentina	Australia	Original	1999	2000
Australia	Samoa	Original	2009	2013
Australia	Marshall Islands	Original	2010	2012

Create Edge List

From this, we can create a data frame of edges (connections)

```
edges <- raw_data %>%
  select(Source = `Country A`, Target = `Country B`, Status) %>%
  filter(!is.na(Source) & !is.na(Target))

kable(head(edges))
```

Source	Target	Status
Australia	Kiribati	In Force
Australia	Philippines	In Force
Australia	Sri Lanka	In Force
Argentina	Australia	In Force
Australia	Samoa	In Force
Australia	Marshall Islands	In Force

Create Node List

And also nodes:

```
nodes <- c(edges$Source, edges$Target) %>%
  unique() %>%
  tibble(id = .) %>%
```

```
  arrange(id)

kable(head(nodes))
```

id
Albania
Algeria
Angola
Argentina
Armenia
Australia

Get Some Geographic Coordinates

To make it mappable, we will need latitude and longitude. I found this list on internet:

```
countries <- read_csv("countries.csv")
country_coords <- countries[, c(3,1,2)]
kable(head(countries))
```

longitude	latitude	COUNTRY	ISO	COUNTRYAFF	AFF_ISO
-170.7007	-14.30571	American Samoa	AS	United States	US
166.6380	19.30205	United States Minor Outlying Islands	UM	United States	US
-159.7877	-21.22261	Cook Islands	CK	New Zealand	NZ
-149.4004	-17.67468	French Polynesia	PF	France	FR
-169.8688	-19.05231	Niue	NU	New Zealand	NZ
-128.3150	-24.36612	Pitcairn	PN	United Kingdom	GB

Aside - data cleaning

```
nodes <- nodes %>%
  mutate(map_name = recode(id,
                           "Cape Verde" = "Cabo Verde",
                           "Brunei" = "Brunei Darussalam",
                           "Curaçao" = "Curacao",
                           "Republic of the Congo" = "Congo",
                           "DR Congo" = "Congo DRC",
                           "Czechia" = "Czech Republic",
                           "Ivory Coast" = "Côte d'Ivoire",
                           "Russia" = "Russian Federation",
                           "São Tomé and Príncipe" = "Sao Tome and Principe",
                           "Korea, Republic of" = "South Korea",
                           "Russian Federation" = "Russia",
                           "Viet Nam" = "Vietnam"))
```

Join our nodes with the coordinates

```
nodes_with_geo <- left_join(nodes, country_coords, by = c("map_name" = "COUNTRY"))

# Build a shorter list of nations
short_list <- c("United States", "China", "Germany", "Japan", "India", "United Kingdom", "Franc

# Filter out countries where we couldn't find a coordinate match
nodes_final <- nodes_with_geo %>% filter(!is.na(longitude) & map_name %in% short_list)

# Filter edges to only include countries we successfully located
edges_final <- edges %>%
  filter(Source %in% nodes_final$id & Target %in% nodes_final$id)
```

Create A Graph Object

From the igraph library we can create a graph object, connecting our nodes and edges, relating size to the degree of connectedness and making it mappable:

```
tax_geo_graph <- graph_from_data_frame(d = edges_final,
                                       vertices = nodes_final,
                                       directed = FALSE)

# Calculate degree for sizing
V(tax_geo_graph)$degree <- degree(tax_geo_graph)

# Create the Map ---
# Define layout using the columns 'longitude' and 'latitude'
layout_geo <- create_layout(tax_geo_graph,
                            layout = 'manual',
                            x = V(tax_geo_graph)$longitude,
                            y = V(tax_geo_graph)$latitude)
```

Build a plot with ggraph

ggraph is the equivalent of ggplot() in ggplot2 but for our specific case. If you are comfortable with ggplot, it is effectively the same:

```
world_map <- map_data("world")

ggraph(layout_geo) +
  # The Map Background
  geom_polygon(data = world_map, aes(x = long, y = lat, group = group),
               fill = "grey90", color = "white", size = 0.2) +

  # The Edges (Use arcs for better visibility on maps)
  geom_edge_arc(aes(color = Status), alpha = 0.3, strength = 0.2) +

  # The Nodes
```

```
  geom_node_point(aes(size = degree), color = "steelblue", alpha = 0.5) +

  # Make it pretty
  scale_size(range = c(1, 4)) +
  coord_fixed(1.3) + # Keeps the map aspect ratio correct
  theme_void() +     # Removes axis labels and grey background
  theme(legend.position = "none") +
  labs(title = "Global Tax Treaty Network")
```

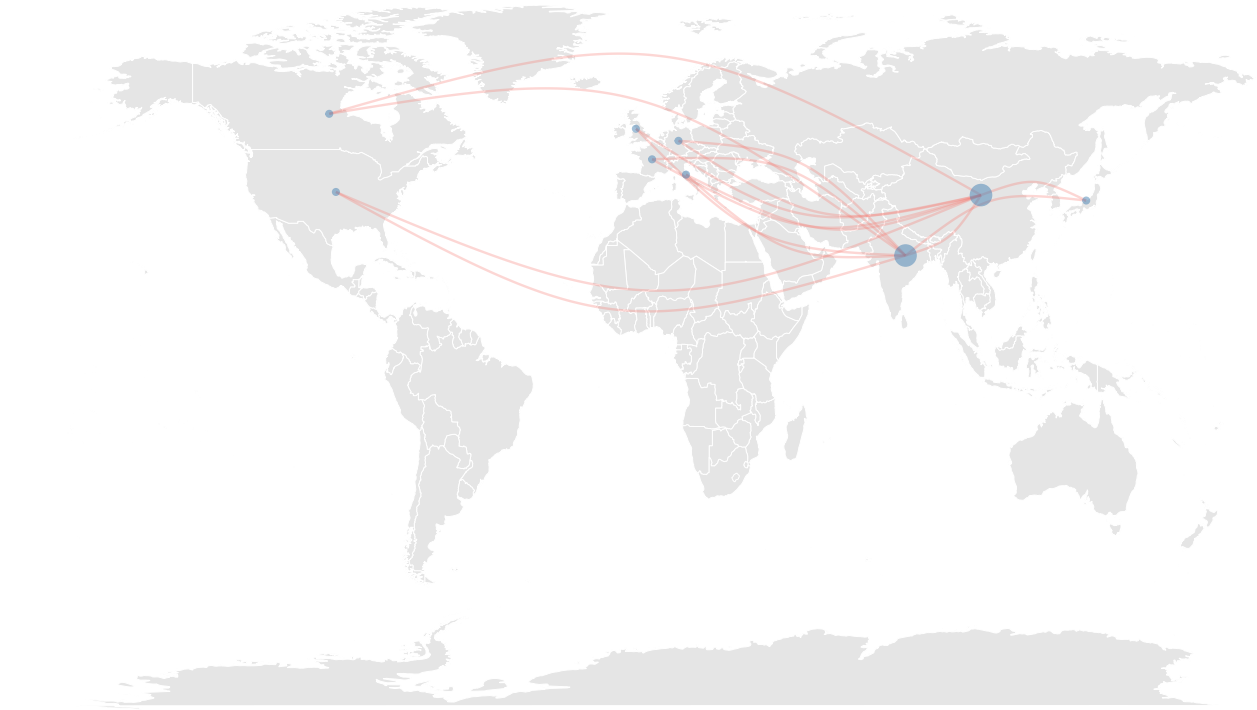


Figure 9: Geospatial visualization of treaty connections

DAG

What is DAG?

DAG means Directed Acyclic Graph. A DAG visualization is a graphical representation of a network where information or processes flow in a strictly one-way direction, with no loops.

- A DAG visualization is a graphical representation of a network where information or processes flow in a strictly one-way direction, with no loops.
- Used to represent pipelines (Data Engineering) and Causal Inference (statistics).

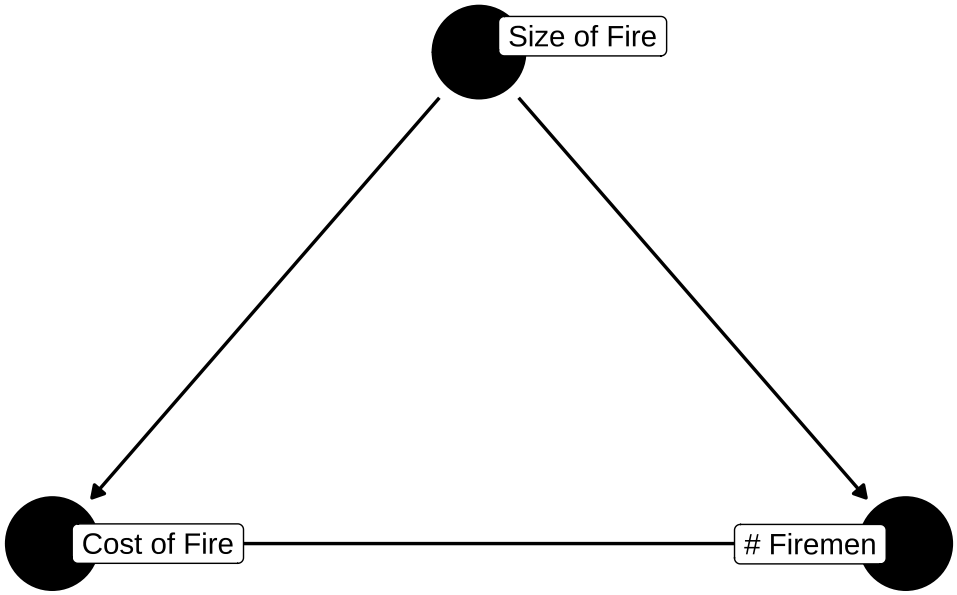
and DAG visualizations are

- Directed - All connections (edges) have a direction, usually shown with arrows (4$A \rightarrow B$)
- Acyclic - The arrows will never end up back where they started. There are no loops.
- Graph - It is a network of nodes (points) and edges (lines).

The "no loop" property makes DAG useful for visualizing dependencies and cause-and-effect.

```
library(ggdag)
library(ggplot2)
library(ggraph)

# Define the relationships
# Syntax: Outcome ~ Cause
dag_model <- dagify(
  Fire_Cost ~ Firemen,  # The specific relationship we want to test
  Fire_Cost ~ Size_of_Fire,       # Size of Fire causes Fire Cost (Confounder -> Outcome)
  Firemen ~ Size_of_Fire,         # Size of Fire causes Firemen (Confounder -> Exposure)

  # Add  labels
  labels = c(
    Fire_Cost = "Cost of Fire",
    Firemen = "# Firemen",
    Size_of_Fire = "Size of Fire"
  ),

  # The experimental variables
  exposure = "Firemen",
  outcome = "Fire_Cost"
)

# Plot the DAG
# text = FALSE removes the node names so we can use the nice labels instead
ggdag(dag_model, text = FALSE, use_labels = "label") +
  theme_dag() +
  ggtitle("Causal Diagram: The Confounder Problem")
```

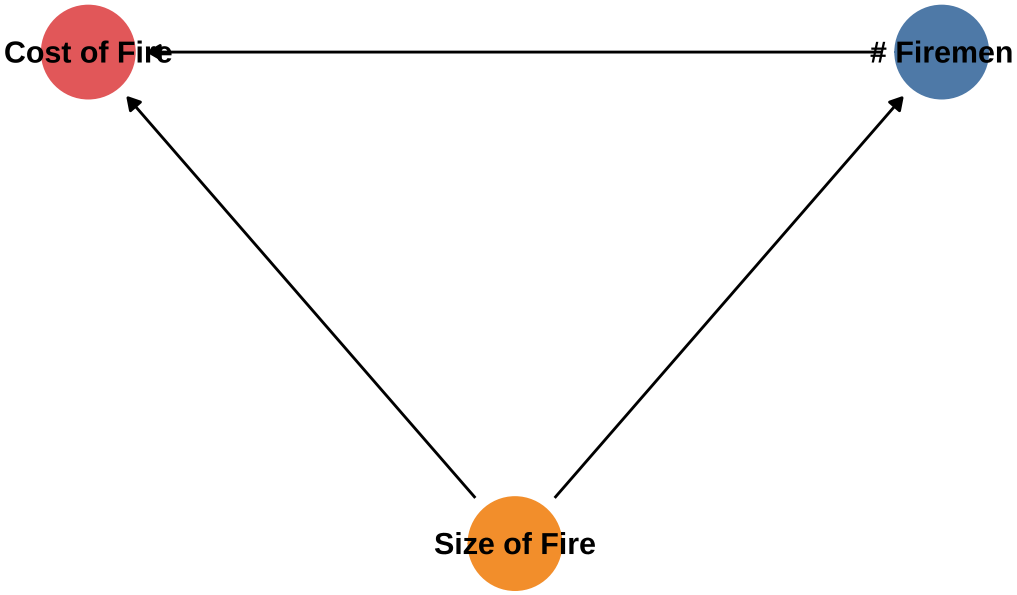
Causal Diagram: The Confounder Problem



ggdag makes ugly visualizations and they are a little tricky to modify. BUT we can convert it to something ggplot can use with tidy_dagitty in ggdag:

```
# Tidy the DAG R object first so we can plot it with ggplot
dag_tidy <- tidy_dagitty(dag_model)

# Plot using regular ggplot
ggplot(dag_tidy, aes(x = x, y = y, xend = xend, yend = yend)) +
  geom_dag_edges(
    edge_width = 0.5
  ) +
  # Draw the nodes and map color to the 'label' column
  geom_dag_point(aes(color = label)) +
  geom_dag_text(
    aes(label = label),
    color = "black"
  ) +
  # Apply any custom theme and colors
  theme_dag(legend.position = "none") +
  scale_color_manual(values = c(
    "Cost of Fire" = "#E15759",
    "# Firemen" = "#4E79A7",
    "Size of Fire" = "#F28E2B"
  )) +
  ggtitle("Causal Diagram: The Confounder Problem")
```

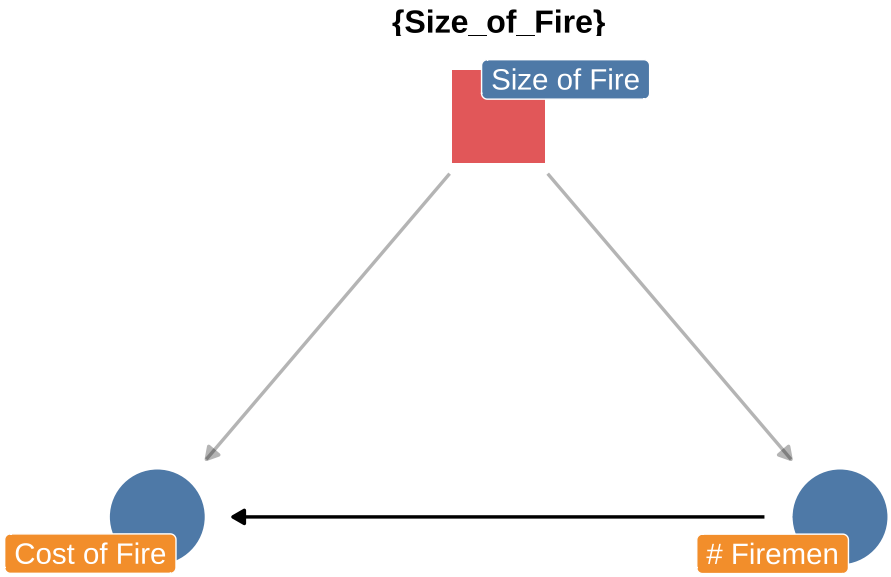
Causal Diagram: The Confounder Problem



A little extra, the real use of DAG is to correct issues like the one we are seeing:

```
ggdag_adjustment_set(dag_model, text = FALSE, use_labels = "label", shadow = TRUE) +
  theme_dag() +
  scale_color_manual(values = c("#E15759", "#4E79A7"), na.value="grey") +

  # Clean up the plot
  theme(legend.position = "none") +
  ggtitle("Adjustment Set: Control for 'Size of Fire'")
```

Adjustment Set: Control for 'Size of Fire'



Takeaways

Theme and Palette

Generating consistent, publication-ready graphics in Quarto/R, regardless of the chart type requires theme and palette changes to get a certain style/look.

The use of Theme in ggplot2

Theme controls all non-data elements. For this report

- **Background:** White, minimal shading.
- **Gridlines:** Few or none
- **Axes:** Few ticks, high data-to-ink ratio (i.e., less ink spent on borders/ticks, more on the data itself).

Use ggplot functions like `theme_tufte()`, `theme_minimal()`, or `theme_light()`, followed by specific changes like removing minor gridlines or axis text to get the right look

The use of the Palette in ggplot2

The **Palette** controls the color mapping:

- Try using inclusive color schemes (e.g., colorblind)
- The `ggthemes` functions like `scale_color_tableau("Tableau 10")` uses the colors recognized as the Tableau standard.

The Visualization Bridge

Even for highly specialized research visualizations like **UMAP** or custom graphs, you typically generate the coordinates or structure first, and then you pipe that data structure into a **ggplot2** function OR use a package built on `ggplot2`, like `ggdag` or `geom_sf`.

The visualization is ultimately rendered by `ggplot2` and the theme and color scales are consistent. This is not perfect because DAG needed modification but UMAP was straightforward.